Clase 7

Memoria distribuida

Programa distribuido: programa concurrente comunicado por mensajes.
Primitivas de pasaje de mensajes: interfaz con el sistema de comunicación.
Los procesos comparten únicamente canales, lógicos o físicos. Clasificación:
Según quien envía y recibe:

- **Mailbox**: canales compartidos conocidos por todos los procesos, todos pueden enviar o recibir.
- **Input port**: un único proceso es el receptor, pero cualquiera puede enviar.
- **Link**: un único proceso envía, y un único proceso recibe.

Según la dirección de comunicación:

- **Unidireccionales**: comunicación en un solo sentido.
- **Bidireccionales**: comunicación en ambos sentidos.

Según sincronicidad:

- **Sincrónicos**: ambos procesos deben estar al mismo tiempo para que se de la comunicación.
- **Asincrónico**: el que envía, envía cuando quiere, el mensaje queda guardado para que el receptor reciba cuando pueda.

Las variables son locales a los procesos
No se requiere un método especial de exclusión mutua
Los procesos interactúan comunicándose a través de primitivas de envío y recepción.

Mecanismos para el Procesamiento Distribuido:

- **PMA**: canales tipo Mailbox, Unidireccionales, Asincrónicos.
- **PMS**: canales tipo Link, Unidireccionales, Sincrónicos.
- **RPC**: canales tipo Link, Bidireccionales, Sincrónicos.
- **Rendezvous**: canales tipo Link, Bidireccionales, Sincrónicos.

Patrones de interacción:

- **Productores y consumidores** (**Filtros o pipes**): procesos filtros que reciben info de otro, la procesan y la devuelven a otro proceso para que continúe trabajando.
- **Clientes y servidores**: los clientes no pueden resolver algo, entonces se lo piden al proceso distinto, Servidor, que lo único que hace es atender y resolver esos pedidos. Mientras no haya pedidos no trabaja. El cliente no trabaja hasta recibir su resultado.
- **Pares que interactúan** (**Peers**): procesos iguales que realizan el mismo trabajo, que cada tanto interactúan para compartir información o sincronizarse.

Pasaje de Mensajes Asincrónicos - PMA

Los procesos comparten únicamente canales, las variables son de cada proceso.
Canales: colas de mensajes que han sido enviados pero todavía no recibidos.Tienen orden FIFO.
Declaración:

- chan nombre_canal(nombre_var: tipo)
- chan acceso_disco(INT cilindro, …)

Los canales son conocidos por todos los procesos.

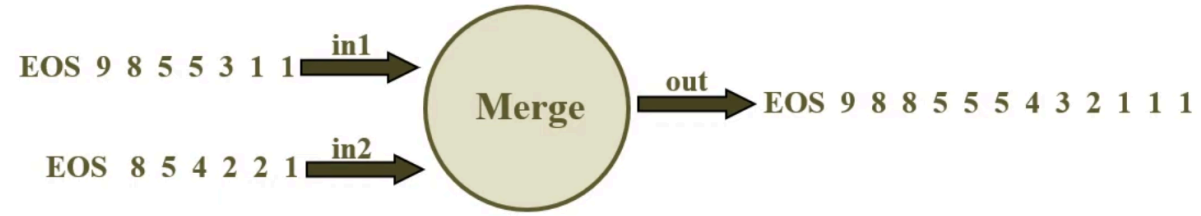
Operaciones

- **Send**: un proceso agrega un mensaje al final de la cola. No bloqueante
    - send nombre_canal(var)
- **Receive**: un proceso recibe un mensaje del canal. Bloquea (se duerme) hasta tomar un mensaje.
    - receive nombre_canal(var)
- Ambas son atómicas. Un canal nunca será usado por más de un proceso a la vez.
- **Empty**: determina si un canal está vacío. Útil cuando se puede realizar otro trabajo mientras se espera un mensaje.
    - empty(nombre_canal)
    - Podría ocurrir que dos procesos pregunten por un mismo canal, habiendo un solo mensaje, dejando a uno de los dos bloqueados cuando haga el receive.

Los canales son de tipo mailbox (todos envían y reciben), pero podrían ser usados de cualquier forma.

Productores y consumidores (filtro) - Red de Ordenacion

Filtro: proceso que recibe mensajes de uno o más canales de entrada y envía mensajes a uno o más canales de salida. La salida es la ejecución del filtro sobre los valores de entrada.



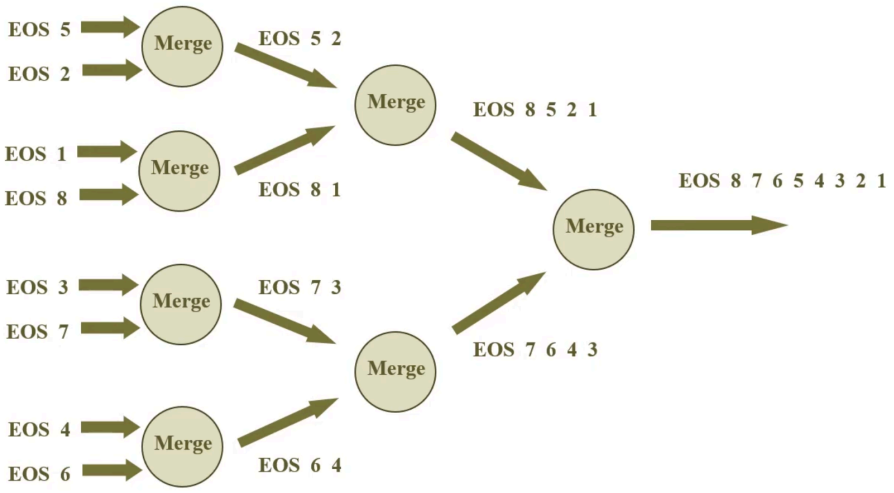
```
chan  in1(int), in2(int), out(int);

Process Merge
{ int v1, v2;

  receive in1(v1);
  receive in2(v2);
  while (v1 ≠ EOS) and (v2 ≠ EOS)
    { if  (v1 ≤ v2) { send out(v1); receive in1(v1); }
      else  { send out(v2); receive in2(v2); }
    }

  if  (v1 == EOS) while (v2 ≠ EOS) {send out(v2); receive in2(v2);}
  else  while (v1 ≠ EOS) {send out(v1); receive in1(v1);}

  send out (EOS);
}
```



Se utilizan n-1 procesos, con un ancho de $\log_2 n$.

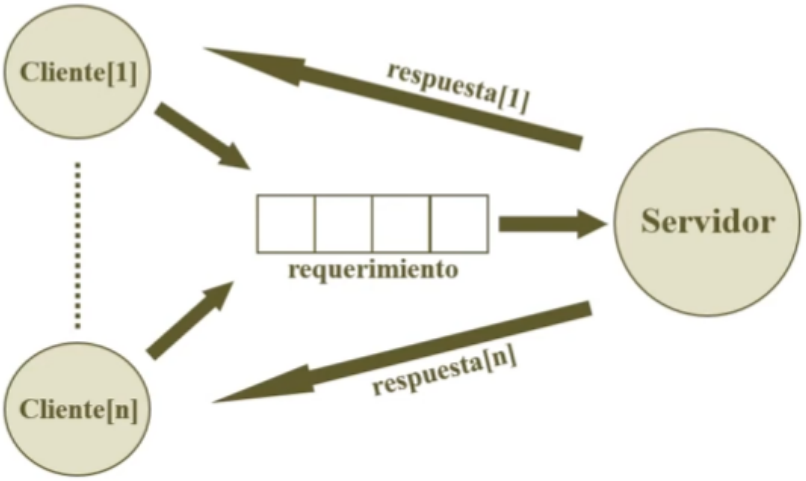
- 8 - 1 = 7 procesos. Cada número estaría en una hoja, y para n hojas se tienen n-1 nodos internos (Merge)
- $\log_2 8 = 3$ de ancho

Clientes y Servidores - Monitores Activos

Servidor: proceso que maneja pedidos de procesos clientes. Vamos a hacer que simulen monitores.

- Monitor: maneja recursos compartidos a través de **variables permanentes** y un conjunto de **procedimientos**.

Cliente: envía un mensaje a un canal general, recibe la respuesta por uno propio.



Simulación de Monitor:

- **Variables permanentes**: variables locales de Servidor
- Llamado a **procedure**: Cliente envía mensaje por el canal requerimiento, recibe resultado por canal propio.

Si se quieren simular múltiples **procedures**, se puede realizar a través de un CASE en el servidor que pregunte por la operación requerida por el Cliente enviada por el canal.

```
chan request(int idCliente, clase_op, tipo_arg);
chan respuesta[n](tipo_result);
```

```
Process Servidor
{  int IdCliente; clase_op oper; tipo_arg args;
   tipo_result resultados;
   código de inicialización;
   while ( true)
      {  receive request(IdCliente, oper, args);
         if ( oper == op1 ) { cuerpo de op1;}
            .......
         elsif ( oper == opn ) { cuerpo de opn;}
         send respuesta[IdCliente](resultados);
      }
}
```

Butterfly Barrier

Barreras Simétrica – *Butterfly barrier*

```
int E = log(N);
int arribo[1:N] = ([N] 0);

process P[i=1..N]
{  int j;
   while (true)
      {  //Sección de código anterior a la barrera.
         //Inicio de la barrera
         for (etapa = 1; etapa <= E; etapa++)
            {  j =  (i-1) XOR (1<<(etapa-1));     //calcula el proceso con cual sincronizar
               while (arribo[i] == 1) → skip;
               arribo[i] = 1;
               while (arribo[j] == 0) → skip;
               arribo[j] = 0;
            }
         //Fin de la barrera
         //Sección de código posterior a la barrera.
      }
}
```

Finales

26/2/2025

PROGRAMACION CONCURRENTE EXAMEN FINAL 26-2-2025
En todos los casos, responda con claridad y sintéticamente. En los casos que corresponda, NO SE CONSIDERARAN RESPUESTAS SIN JUSTIFICAR. Tiempo Máximo 2 hs.
NOTA: Para la aprobación es condición necesaria que el ej 4 o el 5 estén BIEN.

1- ¿En qué consiste la propiedad de "A lo sumo una vez"? ¿Qué efecto tiene sobre las sentencias de un programa concurrente? De ejemplos de sentencias que cumplan y de sentencias que no cumplan con ASV.

2- a) ¿En qué consiste la comunicación guardada y cuál es su utilidad? Describa cómo es la ejecución de sentencias de alternativa e iteración que contienen comunicaciones guardadas.
b) Dado el siguiente bloque de código, indique para cada inciso que valor queda en aux, o si el código queda bloqueado. Justifique sus respuestas.

```
aux = -1;
...
if (A == 0); P2?(aux) -> aux = aux + 2;
□ (A == 1); P3?(aux) -> aux = aux + 5;
□ (B == 0); P3?(aux) -> aux = aux + 7;
end if;
...
```

i) Si el valor de A=1 y B=2 antes del IF, y sólo P2 envía el valor 6.
ii) Si el valor de A=0 y B=2 antes del IF, y sólo P2 envía el valor 8.
iii) Si el valor de A=2 y B=0 antes del IF, y sólo P3 envía el valor 6.
iv) Si el valor de A=2 y B=1 antes del IF, y sólo P3 envía el valor 9.
v) Si el valor de A=1 y B=0 antes del IF, y sólo P3 envía el valor 14.
vi) Si el valor de A=0 y B=0 antes del IF, y P3 envía el valor 9 y P2 el valor 5.

3- a) Defina las métricas de speedup y eficiencia. ¿Cuál es el significado de cada una de ellas (qué miden)? ¿Cuál es el rango de valores posibles de cada uno? Ejemplifique.
b) Defina escalabilidad en sistemas paralelos.
c) Suponga que la solución a un problema es paralelizada sobre *p* procesadores de dos maneras diferentes. En un caso, el speedup (S) está regido por la función S=p-8 y en el otro por la función S=p/3. ¿Cuál de las dos soluciones se comportará más eficientemente al crecer la cantidad de procesadores? Justifique claramente.
d) Suponga que el tiempo de ejecución de un algoritmo secuencial es de 10000 unidades de tiempo, de las cuales sólo el 95% corresponde a código paralelizable. ¿Cuál es el límite en la mejora que puede obtenerse paralelizando el algoritmo? Justifique claramente.

4- Implemente una solución al problema de exclusión mutua distribuida entre N procesos **utilizando un algoritmo token passing con mensajes asincrónicos**

5- Suponga que N procesos poseen inicialmente cada uno un valor. Se debe calcular el promedio de todos los valores y al finalizar la computación todos deben conocer dicho promedio.
a) Describa conceptualmente las soluciones posibles con memoria distribuida para arquitecturas en estrella (centralizada), anillo circular, totalmente conectada y árbol.
b) Implemente al menos 2 de las soluciones.
c) Para cada una de las soluciones (todas), calcule la cantidad de mensajes y el tiempo (**es importante que considere que eventualmente hay operaciones que pueden realizase concurrentemente**).
d) Instancie c) para N=4, N=8, N=16, N=32 y N=64. Analice la performance para cada caso y compare las soluciones.
Nota: puede suponer que cada una de las operaciones tarda una unidad de tiempo.

1. Propiedad ASV

Una sentencia de asignación x = **e** satisface la propiedad de "A lo sumo una vez" si:

1. **e** contiene a lo sumo una referencia crítica y x no es referenciada por otro proceso o
2. **e** no contiene referencias críticas, en cuyo caso x puede ser leída por otro proceso.

Si una sentencia de asignación cumple la propiedad ASV, entonces su ejecución parece atómica, ya que la variable compartida será leída o escrita sólo una vez.
Si una expresión o asignación no satisface ASV con frecuencia es necesario ejecutarla atómicamente.

Ejemplos:

- int x=0, y=0;
  co x=x+1 // y=y+1 oc;
  No hay ref. críticas en ningún proceso.
  En todas las historias ***x = 1*** e ***y = 1***

- int x = 0, y = 0;
  co x=y+1 // y=y+1 oc;
  El 1er proceso tiene 1 ref. crítica. El 2do ninguna.
  Siempre ***y = 1*** y ***x = 1 o 2***

- int x = 0, y = 0;
  co x=y+1 // y=x+1 oc;
  Ninguna asignación satisface ASV.
  Posibles resultados: ***x= 1 e y =2*** / ***x = 2*** e ***y = 1***
  Nunca debería ocurrir x = 1 e ***y = 1 → ERROR***

2.a. Comunicación guardada

La comunicación guardada es una técnica de la programación concurrente donde la comunicación (envío o recepción de mensajes) sólo se ejecuta si se cumplen ciertas condiciones. Es decir, hacer un AWAIT hasta que una condición sea verdadera.
Es útil en sistemas distribuidos porque permite que los procesos esperen mensajes sin bloquearse innecesariamente o tomen decisiones según la disponibilidad de los datos.
Las sentencias de comunicación guardada se ejecutan no deterministicamente.
Formato de las sentencias:

B; C → **S**;

- **B** puede omitirse y se asume true.
- **B** y **C** forman la guarda.
- La guarda tiene éxito si **B** es true y ejecutar **C** no causa demora.
- La guarda se **bloquea** si **B** es **true** pero **C** no puede ejecutarse inmediatamente.
- La guarda **falla** si **B** es **falsa**.

Ejecución:

if B_1; comunicación$_1$ → S_1;
- B_1; comunicación$_1$ → S_1;
fi

Primero, se evalúan las guardas:

1. Si todas las guardas fallan, finaliza el if sin efecto.
2. Si al menos una guarda tiene éxito, se elige una de forma no determinística.
3. Si algunas guardas se bloquean, se espera hasta que alguna de ellas tenga éxito.

Segundo, se ejecuta la sentencia de comunicación de la guarda elegida.

Tercero, se ejecuta la sentencia S_i.

2.b. Ejecución if

I. aux = -1. Entra por la segunda guarda porque sería la única verdadera, pero se queda bloqueada esperando el mensaje de P3.

II. aux = 10. Entra por la primera guarda porque sería la única verdadera, recibe el mensaje de P2 y ejecuta la sentencia.
III. aux = 13. Entra por la tercera guarda porque sería la única verdadera, recibe el mensaje de P3 y ejecuta la sentencia.
IV. aux = No realiza ninguna acción ya que ninguna condición sería verdadera.
V. Como la segunda y la tercera guarda son verdaderas existen dos caminos posibles, se ejecuta uno de forma no determinística.
    A. aux = 19. Entra por la segunda guarda, recibe el mensaje de P3 y ejecuta la sentencia.
    B. aux = 21. Entra por la tercera guarda, recibe el mensaje de P3 y ejecuta la sentencia.
VI. Como la primera y la tercera guarda son verdaderas existen dos caminos posibles, se ejecuta uno de forma no determinística.
    A. aux = 7. Entra por la primera guarda, recibe el mensaje de P2 y ejecuta la sentencia.
    B. aux = 16. Entra por la tercera guarda, recibe el mensaje de P3 y ejecuta la sentencia.

3.a. Speedup - Eficiencia

Speedup

Mide la mejora de rendimiento de una aplicación al aumentar la cantidad de procesadores. Sirve para saber cuánto mejor es la solución paralela respecto a la secuencial.
S es el cociente entre el tiempo de ejecución del algoritmo serial conocido más rápido (T_s) y el tiempo de ejecución paralelo del algoritmo elegido (T_p):

$$S = \frac{T_S}{T_P}$$

S óptimo → Depende de la arquitectura, si es homogénea es la cantidad de procesadores, si es heterogénea es la siguiente fórmula.

$$S_{óptimo} = \sum_{i=0}^{P} \frac{PotenciaCalculo(i)}{PotenciaCalculo(mejor)}$$

Rango de valores: entre 0 y S óptimo.

Eficiencia

Sirve para saber qué tan cerca estamos del Speedup óptimo de nuestra arquitectura, básicamente que tan bien se aprovecha esta última.
Es el cociente entre Speedup y Speedup Óptimo.

$$E = \frac{S}{S_{óptimo}}$$

Rango de valores: entre 0 y 1. 1 corresponde al Speedup perfecto.

3.b.Escalabilidad

Es la capacidad que tiene una solución de poder ser utilizada y funcionar correctamente para problemas o arquitecturas más grandes.

3.c. Problemas paralelizado

De las dos soluciones, la que tiene speedup S = p - 8 se comporta de forma más eficiente a medida que crece el número de procesadores.

- S = p - 1 crece linealmente y se acerca al ideal.
- S = p / 3 crece más lentamente y siempre es un tercio del número de procesadores.

Si analizamos la eficiencia, que se define como:

E = S / p

E = (p - 8) / p -> Esta eficiencia tiende a 1 cuando p crece.
E = (p / 3) / p -> (p/3) / p -> (p * 1)/(3 * p) -> p/3p -> 1/3 -> La eficiencia es constante e igual al 33%, sin importar cuántos procesadores haya.

Por lo tanto, la solución con S = p - 8 es más eficiente, ya que utiliza mejor los procesadores disponibles.

3.d. Límite de mejora

El límite se alcanza cuando se utilizan 9500 procesadores (uno por cada unidad de tiempo paralelizable), lo que reduce el tiempo de ejecución de la parte paralela a 1 unidad de tiempo.

La parte secuencial, que no puede paralelizarse, tarda 500 unidades de tiempo.

T_Paralelo = T_secuencial + T_paralelizable / p

T_Paralelo = 500 + 9500 / 9500 -> 500 + 1 = 501
Speedup = T_secuencial_total / T_Paralelo = 10000 / 501 ≈ 19.96

Este resultado confirma la Ley de Amdahl, la cual establece que el límite de paralelización de un algoritmo no depende de cuántos procesadores se usen, sino de cuánta parte del código es secuencial.

4. Token passing

```
chan token[n]() ;                    # para envío de tokens
chan enter[n](), go[n](), exit[n](); # para comunicación proceso-helper

process helper[i = 1..N] {
    while(true){
        receive token[i]();              # recibe el token
        if(!(empty(enter[i]))){         # si su proceso quiere usar la SC
            receive enter[i]();
            send go[i]();               # le da permiso y lo espera a que termine
            receive exit[i]();
        }
        send token[i MOD N + 1]();      # y lo envía al siguiente cíclicamente
    }
}

process user[i = 1..N] {
    while(true){
        send enter[i]();
        receive go[i]();
        ... sección crítica ...
        send exit[i]();
        ... sección no crítica ...
    }
}
```

> Se asume que el primero ya tenga el token

5. Calcular promedio

5.a.c. Conceptualmente y cantidad de mensajes

Estrella (centralizada)

Cada proceso envía su valor al coordinador. Este los suma, divide por N para obtener el promedio y lo reenvía a todos.
Cantidad de mensajes: 2(N - 1).
Paralelismo solo en el envío inicial. El coordinador es cuello de botella y único punto de falla.

Anillo circular

Un proceso (ej. P0) inicia la suma parcial pasando su valor al siguiente. Cada proceso suma y reenvía. Al completar la vuelta, se divide entre N para obtener el promedio, que luego se difunde por el anillo.
Cantidad de mensajes: (2N) - 1.
No hay paralelismo: cada proceso espera al anterior.

Totalmente conectada

Cada proceso envía su valor a todos los demás. Luego calcula localmente el promedio.
Cantidad de mensajes: N(N - 1).
Alta velocidad gracias al paralelismo completo, pero alto costo en comunicación.

Árbol

Se construye un árbol donde cada nodo envía su valor a su padre (bottom-up) para sumar.
La raíz calcula el promedio dividiendo la suma por N, y luego lo difunde (top-down).
Cantidad de mensajes: 2(N - 1).
Hay cierto grado de paralelismo en ambas etapas (varios hijos pueden enviar al mismo tiempo).

5.b. Implementaciones

Estrella (centralizada)

```
chan canal_valores(INT);
chan canal_promedios[n](REAL promedio);

process Coordinador[0]{              // Coordinador
    INT valor_local;
    INT suma_total  = valor_local;
    REAL promedio_global;

    for (i = 1 to n - 1) {
        INT valor_recibido;
        receive canal_valores(valor_recibido);
        suma = suma + valor_recibido;
    }
    promedio_global  = suma_total / n;
    for (i = 1 to n - 1)
        send canal_promedios[i](promedio_global);
}

process ProcesoTrabajador[i = 1 to n - 1]{ // Cada proceso envía su valor y recibe el promedio
    INT valor_local;
    REAL promedio_recibido;

    send canal_valores(valor_local);
    receive canal_promedios[i](promedio_recibido);
}
```

Anillo circular

```
chan canal_suma[n](INT suma_parcial);      // Para pasar sumas parciales en el anillo
chan canal_promedio[n](REAL promedio_global);  // Para difundir el promedio calculado

process ProcesoInicial[0] {
    INT valor_local
    INT suma_total  = valor_local;
    REAL promedio_calculado;

    // Fase 1
    send canal_suma[1](suma_total);

    //Fase 2
    receive canal_suma[0](suma_total);

    promedio_calculado  = suma_total / n;
    send canal_promedio[1](promedio_calculado);
}

process ProcesoAnillo[i = 1 to n - 1] {
    INT valor_local,
    INT suma_total;
    REAL promedio_recibido;
    INT siguiente = (i + 1) mod n; // El siguiente proceso en el anillo

    // Fase 1: recibe suma parcial, acumula y reenvía
    receive canal_suma[i](suma_total);
    suma_total = suma_total + v;

    send canal_suma[siguiente](suma_total);

    // Fase 2: recibe el promedio y lo reenvía
    receive canal_promedio[i](promedio_recibido);
    send canal_promedio[siguiente](promedio_recibido);
}
```

Totalmente conectada

```
chan canal_valores[n](INT valor_recibido); // Canal para intercambio de valores entre todos los procesos

process ProcesoSimetrico[i = 0 to n - 1] {
    INT valor_local  = ...;
    INT suma_total  = valor_local ;
    REAL promedio_local;

    // Envío mi valor_local a todos los demás procesos
    for (int destino = 0 to n - 1 st destino  <> i)
        send canal_valores[k](valor_local );

    // Recibo los valores de los demás procesos y acumulo la suma
    for (int origen = 0 to n - 1 st origen <> i) {
        INT valor_recibido
        receive canal_valores[i](valor_recibido);
        suma_total  = suma_total  + valor_recibido;
    }
    // Calculo el promedio de todos los valores
    promedio_local = suma_total  / n;
}
```

5.d Analisis performance

Estrella (centralizada)

Fórmula de mensajes: 2(n - 1) sin broadcast , n con broadcast
Fórmula de tiempo: Envío (1) + procesamiento (1) + difusión (1) = 3

N = 4 → Mensajes: 2(4 - 1) = 6, Tiempo: 3
N = 8 → Mensajes: 2(8 - 1) = 14, Tiempo: 3
N = 16 → Mensajes: 2(16 - 1) = 30, Tiempo: 3
N = 32 → Mensajes: 2(32 - 1) = 62, Tiempo: 3
N = 64 → Mensajes: 2(64 - 1) = 126, Tiempo: 3

Anillo circular

Fórmula de mensajes: 2(n - 1) (una vuelta para la suma + una vuelta para difusión)
Fórmula de tiempo: 2(n - 1) (no hay paralelismo)

N = 4 → Mensajes: 2(4 - 1) = 6, Tiempo: 6
N = 8 → Mensajes: 2(8 - 1) = 14, Tiempo: 14
N = 16 → Mensajes: 2(16 - 1) = 30, Tiempo: 30
N = 32 → Mensajes: 2(32 - 1) = 62, Tiempo: 62
N = 64 → Mensajes: 2(64 - 1) = 126, Tiempo: 126

Totalmente conectada

Fórmula de mensajes: n(n - 1) sin broadcast, n con broadcast
Fórmula de tiempo: Envío + recepción, ambos en paralelo = 2

N = 4 → Mensajes: 4 × (4 - 1) = 12, Tiempo: 2
N = 8 → Mensajes: 8 × (8 - 1) = 56, Tiempo: 2
N = 16 → Mensajes: 16 × (16 - 1) = 240, Tiempo: 2
N = 32 → Mensajes: 32 × (32 - 1) = 992, Tiempo: 2
N = 64 → Mensajes: 64 × (64 - 1) = 4032, Tiempo: 2

Árbol

Fórmula de mensajes: 2(n - 1) (una subida bottom-up + una bajada top-down)
Fórmula de tiempo: $2 \times \log_2(n)$ (dos recorridos del árbol)
Resultados:

N = 4 → Mensajes: 2(4 - 1) = 6, Tiempo: 2 × 2 = 4
N = 8 → Mensajes: 2(8 - 1) = 14, Tiempo: 2 × 3 = 6
N = 16 → Mensajes: 2(16 - 1) = 30, Tiempo: 2 × 4 = 8
N = 32 → Mensajes: 2(32 - 1) = 62, Tiempo: 2 × 5 = 10
N = 64 → Mensajes: 2(64 - 1) = 126, Tiempo: 2 × 6 = 12

12/2/2025

PROGRAMACION CONCURRENTE EXAMEN FINAL 12-2-2025

En todos los casos, responda con claridad y sintéticamente. En los casos que corresponda, NO SE CONSIDERARAN RESPUESTAS SIN JUSTIFICAR. Tiempo Máximo 2 hs.

NOTA: Para la aprobación es condición necesaria que el ej 4 o el 5 estén BIEN.

1- Defina programa concurrente, programa paralelo y programa distribuido.

2- a) ¿En qué consiste la comunicación guardada y cuál es su utilidad? Describa cómo es la ejecución de sentencias de alternativa e iteración que contienen comunicaciones guardadas.

b) Dado el siguiente bloque de código, indique para cada uno de los ítems si son equivalentes o no. Justificar cada caso (de ser necesario dar ejemplos).

Segmento 1	Segmento 2

...
int cant=1000;
While (true)
{ IF (cant > 15); datos?(cant) → *Sentencias1*
□ (cant < 5); datos?(cant) →*Sentencias2*
□ (**INCOGNITA**); datos?(cant) → *Sentencias3*
END IF
} ...
	...
	int cant=1000;
	DO (cant > 15); datos?(cant) → *Sentencias1*
	□ (cant < 5); datos?(cant) → *Sentencias2*
	□ (**INCOGNITA**); datos?(cant) →
	Sentencias3
	END DO
	...

i. ***INCOGNITA*** equivale a: ***(cant = 5) or (cant = 15)***

ii. ***INCOGNITA*** equivale a: ***(cant > 0)***

iii. ***INCOGNITA*** equivale a: ***((cant >= 2) and (cant <= 20))***

iv. ***INCOGNITA*** equivale a: ***((cant > 5) and (cant <= 15))***

v. ***INCOGNITA*** equivale a: ***((cant > 5) and (cant < 15))***

3- a) Defina las métricas de speedup y eficiencia. ¿Cuál es el significado de cada una de ellas (qué miden)? ¿Cuál es el rango de valores posibles de cada uno? Ejemplifique.

b) Suponga que la solución a un problema es paralelizada sobre *p* procesadores de dos maneras diferentes. En un caso, el speedup (S) está regido por la función S=p/3 y en el otro por la función S=p-8. ¿Cuál de las dos soluciones se comportará más eficientemente al crecer la cantidad de procesadores? Justifique claramente.

c) Suponga que el tiempo de ejecución de un algoritmo secuencial es de 10000 unidades de tiempo, de las cuales sólo el 95% corresponde a código paralelizable. ¿Cuál es el *límite en la mejora* que puede obtenerse paralelizando el algoritmo? Justifique claramente.

4- Implemente una solución al problema de exclusión mutua distribuida entre N procesos **utilizando un algoritmo token passing con mensajes asincrónicos**

5- Sea la siguiente solución al problema del producto de matrices de nxn con P procesos en paralelo con variables compartidas.

```
process worker [w = 1 to P] { # strips en paralelo (p strips de n/P filas)
  int first = (w-1) * n/P + 1        # Primera fila del strip
  int last = first + n/P - 1;        # Ultima fila del strip
  for [i = first to last] {
     for [j = 1 to n] {
          c[i,j] = 0.0;
          for [k = 1 to n]
            c[i,j] = c[i,j] + a[i,k]*b[k,j];
       }
     }
}
```

Suponga n=256 y cada procesador capaz de ejecutar un proceso.

a) Calcular cuántas asignaciones, sumas y productos se hacen secuencialmente (caso en que P=1), y cuántas se realizan en cada procesador en la solución paralela con P=4.

b) Si los procesadores P1 a P3 son idénticos, con tiempos de asignación 1, de suma 2 y de producto 3, y si el procesador P4 es 3 veces más lento, calcule cuánto tarda el proceso total concurrente

c) ¿Cuál es el valor del speedup?

d) ¿Cómo modificaría el código para lograr un mejor speedup?

NOTA: para realizar los cálculos no tenga en cuenta las operaciones de asignación e incremento correspondientes a las sentencias *for*.

1. Define programas

Concurrente

Un programa concurrente consiste en un conjunto de tareas o procesos secuenciales que pueden ejecutarse intercalándose en el tiempo y que cooperan para resolver un problema. Básicamente, es un concepto de software que dependiendo de la arquitectura subyacente da lugar a las definiciones de programación paralela o distribuida.
Un programa concurrente es un programa que tiene más de un hilo de ejecución o flujo de control, es decir, está dividido en tareas que pueden ejecutarse en paralelo o simultáneamente. No depende de un número determinado de procesadores ni de una arquitectura particular.

Paralelo

La programación paralela consiste en un programa concurrente que se ejecuta sobre múltiples procesadores que pueden tener una memoria compartida y que son utilizados para incrementar la performance de un programa concurrente.

Distribuido

La programación distribuida es un caso especial de la programación concurrente en la que se cuenta con varios procesadores pero no se posee una memoria compartida. Los procesos se interconectan por una red de comunicaciones, para interactuar se comunican mediante pasaje de mensajes.

2.a. Comunicación guardada

La comunicación guardada permite esperar más que un solo mensaje, a través de hacer un AWAIT hasta que una condición sea verdadera. Esto es, tener la posibilidad de poder esperar varios mensajes por distintos canales, al mismo tiempo, sin saber el orden en que van a llegar.
Es útil para que, de forma no determinística, se puedan recibir distintos tipos de mensajes en una misma sección de código.

Ejecución:

if B_1; comunicación$_1$ → S_1;
- B_1; comunicación$_1$ → S_1;
fi

Primero, se evalúan las guardas:

4. Si todas las guardas fallan, finaliza el if sin efecto.
5. Si al menos una guarda tiene éxito, se elige una de forma no determinística.
6. Si algunas guardas se bloquean, se espera hasta que alguna de ellas tenga éxito.

Segundo, se ejecuta la sentencia de comunicación de la guarda elegida.

Tercero, se ejecuta la sentencia S_i.

2.b. Incógnita

I. (cant = 5) or (cant = 15)
En ambos casos quedan casos sin cubrir, 5 < cant < 15. En el segmento 1 el código podría quedar loopeando sin entrar por ninguna guarda, ya que ninguna sería verdadera. Por otro lado, en el segmento 2 si ocurriera que ninguna guarda cumple, simplemente termina el DO y la ejecución del código avanza.

II. (cant > 0)
En ambos casos quedan todos los casos posibles cubiertos, por lo que ambos programas quedarían ejecutándose en loop.

III. ((cant >= 2) and (cant <= 20))
En ambos casos quedan todos los casos posibles cubiertos, por lo que ambos programas quedarían ejecutándose en loop.

IV. ((cant > 5) and (cant <= 15))
Ningún caso contempla cuando cant = 5, por lo que ocurre la misma problemática que en el caso 1. En esa situación, el segmento 1 quedaría ejecutándose en loop sin parar, mientras que el segmento 2 termina la ejecución del DO debido a que no hay guardas verdaderas y continúa la ejecución del código.

V. ((cant > 5) and (cant < 15))
Ningún caso contempla cuando cant = 5 o cant = 15, por lo que ocurre lo mismo que en el caso anterior. En esa situación, el segmento 1 quedaría ejecutándose en loop sin parar, mientras que el segmento 2 termina la ejecución del DO debido a que no hay guardas verdaderas y continúa la ejecución del código.

3.a. Speedup - Eficiencia

Speedup

Mide la mejora de rendimiento de una aplicación al aumentar la cantidad de procesadores. Sirve para saber cuánto mejor es la solución paralela respecto a la secuencial.
S es el cociente entre el tiempo de ejecución del algoritmo serial conocido más rápido (T_s) y el tiempo de ejecución paralelo del algoritmo elegido (T_p):

$$S = \frac{T_S}{T_P}$$

S óptimo → Depende de la arquitectura, si es homogénea es la cantidad de procesadores, si es heterogénea es la siguiente fórmula.

$$S_{óptimo} = \sum_{i=0}^{P} \frac{PotenciaCalculo(i)}{PotenciaCalculo(mejor)}$$

Rango de valores: entre 0 y S óptimo.

Eficiencia

Sirve para saber qué tan cerca estamos del Speedup óptimo de nuestra arquitectura, básicamente que tan bien se aprovecha esta última. Nos permite independizarnos de la arquitectura.
Es el cociente entre Speedup y Speedup Óptimo.

$$E = \frac{S}{S_{óptimo}}$$

Rango de valores: entre 0 y 1. 1 corresponde al Speedup perfecto.

3.b. Problemas paralelizado

De las dos soluciones, la que tiene speedup S = p - 8 se comporta de forma más eficiente a medida que crece el número de procesadores.

- S = p - 1 crece linealmente y se acerca al ideal.
- S = p / 3 crece más lentamente y siempre es un tercio del número de procesadores.

Si analizamos la eficiencia, que se define como:

E = S / p

E = (p - 8) / p -> Esta eficiencia tiende a 1 cuando p crece.
E = (p / 3) / p -> (p/3) / p -> (p * 1)/(3 * p) -> p/3p -> 1/3 -> La eficiencia es constante e igual al 33%, sin importar cuántos procesadores haya.

Por lo tanto, la solución con S = p - 8 es más eficiente, ya que utiliza mejor los procesadores disponibles.

3.c. Límite de mejora

El límite se alcanza cuando se utilizan 9500 procesadores (uno por cada unidad de tiempo paralelizable), lo que reduce el tiempo de ejecución de la parte paralela a 1 unidad de tiempo.

La parte secuencial, que no puede paralelizarse, tarda 500 unidades de tiempo.

T_Paralelo = T_secuencial + T_paralelizable / p

T_Paralelo = 500 + 9500 / 9500 -> 500 + 1 = 501
Speedup = T_secuencial_total / T_Paralelo = 10000 / 501 ≈ 19.96

Este resultado confirma la Ley de Amdahl, la cual establece que el límite de paralelización de un algoritmo no depende de cuántos procesadores se usen, sino de cuánta parte del código es secuencial.

4. Token passing

```
chan token[n]() ;                    # para envío de tokens
chan enter[n](), go[n](), exit[n](); # para comunicación proceso-helper

process helper[i = 1..N] {
    while(true){
        receive token[i]();                # recibe el token
        if(!(empty(enter[i]))){           # si su proceso quiere usar la SC
            receive enter[i]();
            send go[i]();                 # le da permiso y lo espera a que termine
            receive exit[i]();
        }
        send token[i MOD N + 1]();        # y lo envía al siguiente cíclicamente
    }
}

process user[i = 1..N] {
    while(true){
        send enter[i]();
        receive go[i]();
        ... sección crítica ...
        send exit[i]();
        ... sección no crítica ...
    }
}
```

> Se asume que el primero ya tenga el token

5. Producto de matrices

n = 256

5.a. P = 1 y P = 4

P = 1
Asignaciones: 256^3 + 256^2 → 16.777.216 + 65.536 → 16.842.752
Sumas: 256^3 → 16.777.216
Productos: 256^3 → 16.777.216

P = 4
Strip: 256 / 4 = 64
Asignaciones: 64 * 256^2 + 64 * 256 → 4.194.304 + 16.384 → 4.210.688
Sumas: 64 * 256^2 → 4.194.304
Productos: 64 * 256^2 → 4.194.304

5.b. Tiempo total

Tiempo P1, P2, P3: 4.210.688 + 4.194.304 * 2 + 4.194.304 * 3 =
4.210.688 + 8.388.608 + 12.582.912 = 25.182.208

Tiempo P4: 25.182.208 * 3 = 75.546.624

Tiempo total: 75.546.624

5.c. Speedup

Speedup = Tiempo secuencial / Tiempo paralelo

Tiempo secuencial = 16.842.752 + 16.777.216 * 2 + 16.777.216 * 3
= 16.842.752 + 33.554.432 + 50.331.648
= 100.728.832

Tiempo paralelo = 75.546.624

Speedup = 100.728.832 / 75.546.624 = 1,33

5.d. Mejor speedup

Para mejorar el speedup se debe balancear la carga de trabajo. Esto lo logramos dando a P4 un strip más pequeño sobre el que trabajar. Si a P1, P2 y P3 les damos 77 filas a cada uno, P4 queda trabajando con 25.

P1, P2, P3:
Asignaciones: 77 * 256^2 + 77 * 256 = 5.046.272 + 19.712 = 5.065.984
Sumas: 77 * 256^2 = 5.046.272
Productos: 77 * 256^2 = 5.046.272

Tiempo total = 5.065.984 + 5.046.272 * 2 + 5.046.272 * 3 =
5.065.984 +10.092.544 + 15.138.816 = 30.297.344

P4:
Asignaciones: 25 * 256^2 + 25 * 256 = 1.638.400 + 6.400 = 1.644.800
Sumas: 25 * 256^2 = 1.638.400
Productos: 25 * 256^2 = 1.638.400

Tiempo total = (1.644.800 + 1.638.400 * 2 + 1.638.400 * 3) * 3 =
(1.644.800 +3.276.800 + 4.915.200) * 3 =
9.836.800 * 3 = 29.510.400

Calculamos el speedup con el tiempo paralelo más grande:

Speedup = 100.728.832 / 30.297.344 = 3,32